Job Portal Project Report

1) Problem Statement

Problem

In many places, hiring work is not in one system:

- Job seekers search jobs in one app, but track applications in another place.
- Recruiters post jobs, but managing applicants is not smooth.
- Admin control (role change, company verify) is missing in many small projects.

Why This System Is Needed

This Job Portal gives one single platform where:

- Job seekers can find jobs, save jobs, and apply directly.
- Recruiters can create company profile, post jobs, and manage hiring.
- Admin can verify companies and manage user roles.

So complete hiring process becomes easy, clean, and trustworthy.

---

2) System Architecture

Architecture in Simple Words (Non-Technical View)

- The **user** opens website and does action (search, apply, post job).
- The **frontend** sends request to backend server.
- The **backend** checks login/role and does required work.
- The **database** stores all records and gives data back.
- Then frontend shows final result to user.

Flowchart Shape Guide

- **Oval/Pill** = Start or End
- **Rectangle** = Process step
- **Diamond** = Decision (Yes/No)
- **Parallelogram** = Input/Output (data coming/going)

Easy Architecture Flowchart

```
flowchart LR
  A([Start: User opens website]) --> B[/Input: User action/]
  B --> C[Process: Frontend sends API request]
  C --> D[Process: Backend validates and handles request]
  D --> E[(Database)]
  E --> F[/Output: Response data/]
  F --> G[Process: Frontend updates page]
  G --> H([End: User sees result])

  classDef startend fill:#c7f9cc,stroke:#2d6a4f,stroke-width:2px,color:#1b4332;
```

```
  classDef io fill:#fde68a,stroke:#b45309,stroke-width:2px,color:#7c2d12;
  classDef process fill:#bfdbfe,stroke:#1d4ed8,stroke-width:2px,color:#1e3a8a;
  classDef db fill:#ddd6fe,stroke:#6d28d9,stroke-width:2px,color:#4c1d95;
  class A,H startend;
  class B,F io;
  class C,D,G process;
  class E db;
```

High-Level Components

- **Frontend**: React (Vite), Tailwind-based UI, React Router for SPA navigation.
- **Backend**: Node.js + Express REST API (`/api/v1/...`).
- **Database**: MongoDB with Mongoose models (`User`, `Company`, `Job`, `Application`, `SavedJob`).
- **Auth**: JWT + refresh token flow, HTTP-only cookies, role-based authorization.

Architecture Diagram

```
flowchart LR
  U[User Browser] --> FE[React Frontend]
  FE -->|HTTP/JSON + Cookies| BE[Express Backend API]
  BE --> DB[(MongoDB)]
  BE --> FS[(Static/Public Assets)]

  subgraph Frontend
    FE1[Pages & Components]
    FE2[API Client]
    FE3[Auth State / Context]
  end
  FE --> FE1
  FE --> FE2
  FE --> FE3

  subgraph Backend
    B1[Routes]
    B2[Controllers]
    B3[Middleware: Auth, Rate Limiting, Validation]
    B4[Mongoose Models]
  end
  BE --> B1 --> B2 --> B4 --> DB
  B1 --> B3
```

Interaction Summary

1. Frontend calls backend APIs (GET/POST/PUT/DELETE).
2. Backend validates auth/role via middleware.
3. Controllers execute business logic and read/write MongoDB via Mongoose models.
4. JSON responses are returned to frontend and rendered in role-specific views.

Request-Response Flow (Step by Step)

```
flowchart TD
  S1([Start]) --> S2[/Input: User clicks Apply Job/]
  S2 --> S3[Process: Frontend sends request]
  S3 --> S4{Decision: User authorized?}
  S4 -- Yes --> S5[Process: Save or fetch data]
  S5 --> S6[/Output: Success response/]
  S4 -- No --> S7[/Output: Error/Unauthorized message/]
  S6 --> S8[Process: UI refresh with new data]
  S7 --> S8
  S8 --> S9([End])
```

```
  classDef startend fill:#c7f9cc,stroke:#2d6a4f,stroke-width:2px,color:#1b4332;
  classDef io fill:#fde68a,stroke:#b45309,stroke-width:2px,color:#7c2d12;
  classDef process fill:#bfdbfe,stroke:#1d4ed8,stroke-width:2px,color:#1e3a8a;
  classDef decision fill:#fecaca,stroke:#b91c1c,stroke-width:2px,color:#7f1d1d;
  class S1,S9 startend;
  class S2,S6,S7 io;
  class S3,S5,S8 process;
  class S4 decision;
```

---

3) URI Design / User Flow Diagram

A. Frontend Routes (User Navigation)

Route	Purpose	Access
`/`	Home page, featured jobs/companies	Public
`/auth`	Login/Register	Public
`/jobs`	Job listings + filters	Public
`/jobs/:jobId`	Job details + apply/save	Public (apply/save needs login)
`/jobs/new`	Create job	Protected (Recruiter/Admin)
`/jobs/:jobId/edit`	Edit job	Protected (Owner Recruiter/Admin)
`/companies`	Company listing	Public
`/companies/:companyId`	Company detail + related jobs	Public
`/companies/new`	Create company	Protected (Recruiter/Admin)
`/companies/:companyId/edit`	Edit company	Protected (Owner Recruiter/Admin)
`/dashboard`	Role-based dashboard	Protected
`/404`	Not found page	Public

B. Backend Resource URI Design (Core)

- Auth: `/api/v1/auth/...`
- Users: `/api/v1/users/...`
- Jobs: `/api/v1/jobs/...`
- Companies: `/api/v1/companies/...`
- Applications: `/api/v1/applications/...`
- Saved Jobs: `/api/v1/saved-jobs/...`
- Admin: `/api/v1/admin/...`
- Health: `/api/v1/healthcheck`

C. User Flow Diagram

```
flowchart TD
  A([Start: Visitor opens Home]) --> B{Decision: Logged in?}
  B -- No --> C[Process: Browse jobs/companies]
  C --> D[/Input: Click Sign in/]
  D --> E[Process: Register/Login]
  E --> F[Process: Email verify + session create]
  F --> G[Process: Open Dashboard]

  B -- Yes --> G
  G --> H{Decision: Which role?}
  H -- Jobseeker --> I[Process: Save/apply/track applications]
```

```
  H -- Recruiter --> J[Process: Create company/post jobs/manage pipeline]
  H -- Admin --> K[Process: Verify companies/manage roles]
  I --> L([End])
  J --> L
  K --> L

  classDef startend fill:#c7f9cc,stroke:#2d6a4f,stroke-width:2px,color:#1b4332;
  classDef io fill:#fde68a,stroke:#b45309,stroke-width:2px,color:#7c2d12;
  classDef process fill:#bfdbfe,stroke:#1d4ed8,stroke-width:2px,color:#1e3a8a;
  classDef decision fill:#fecaca,stroke:#b91c1c,stroke-width:2px,color:#7f1d1d;
  class A,L startend;
  class D io;
  class C,E,F,G,I,J,K process;
  class B,H decision;
```

D. Role-Based Flow (Easy to Understand)

```
flowchart LR
  U([Start: Login User]) --> R{Decision: User role?}
  R -->|Jobseeker| JS[Process: Search, save, apply jobs]
  R -->|Recruiter| RC[Process: Create company, post jobs, manage applications]
  R -->|Admin| AD[Process: Verify company, change user role]
  JS --> X([End])
  RC --> X
  AD --> X

  classDef startend fill:#c7f9cc,stroke:#2d6a4f,stroke-width:2px,color:#1b4332;
  classDef process fill:#bfdbfe,stroke:#1d4ed8,stroke-width:2px,color:#1e3a8a;
  classDef decision fill:#fecaca,stroke:#b91c1c,stroke-width:2px,color:#7f1d1d;
  class U,X startend;
  class JS,RC,AD process;
  class R decision;
```

---

4) Database Design

Collections and Key Fields

1. `users`

- `_id`
- `username` (unique)
- `email` (unique)
- `password` (hashed)
- `role` (`jobseeker` | `recruiter` | `admin`)
- `isEmailVerified`
- `refreshToken`
- `forgotPasswordToken`, `forgotPasswordExpiry`
- `emailVerificationToken`, `emailVerificationExpiry`
- `avatar.url`, `avatar.localPath`
- `createdAt`, `updatedAt`

2. `companies`

- `_id`
- `owner` (ref `users._id`)
- `name`

- logo.url, logo.localPath
- website
- location
- description
- isVerified
- createdAt, updatedAt

3. jobs

- _id
- company (ref companies._id)
- title
- description
- skillsRequired[]
- jobType (Full-Time | Part-Time | Internship | Contract)
- experienceLevel (Fresher | Junior | Mid | Senior)
- salaryRange.min, salaryRange.max
- location
- isRemote
- lastDateToApply
- isActive
- createdAt, updatedAt

4. applications

- _id
- job (ref jobs._id)
- applicant (ref users._id)
- resume.url, resume.localPath
- coverLetter
- status (Applied | Shortlisted | Rejected | Hired)
- createdAt, updatedAt

5. savedjobs

- _id
- job (ref jobs._id)
- user (ref users._id)
- Unique compound index: (job, user)
- createdAt, updatedAt

ER Diagram

```
erDiagram
  USER ||--o{ COMPANY : owns
  COMPANY ||--o{ JOB : posts
  USER ||--o{ APPLICATION : submits
  JOB ||--o{ APPLICATION : receives
  USER ||--o{ SAVED_JOB : bookmarks
  JOB ||--o{ SAVED_JOB : bookmarked_in

  USER {
    ObjectId _id
    string username
    string email
```

```
    string role
    bool isEmailVerified
  }

  COMPANY {
    ObjectId _id
    ObjectId owner
    string name
    string website
    string location
    bool isVerified
  }

  JOB {
    ObjectId _id
    ObjectId company
    string title
    string jobType
    string experienceLevel
    bool isRemote
    date lastDateToApply
  }

  APPLICATION {
    ObjectId _id
    ObjectId job
    ObjectId applicant
    string status
  }

  SAVED_JOB {
    ObjectId _id
    ObjectId user
    ObjectId job
  }
```

---

5) REST API Endpoints

Base URL: `/api/v1`

A. Health

Method	Endpoint	Description
GET	`/healthcheck`	Check API/server status

B. Auth

Method	Endpoint	Description
POST	`/auth/register`	Register new user
POST	`/auth/login`	Login user
GET	`/auth/current-user`	Get logged-in user profile
POST	`/auth/logout`	Logout user
POST	`/auth/refresh-token`	Refresh access token
GET/POST	`/auth/verify-email/:token`	Verify email by token
POST	`/auth/resend-verification-email-public`	Resend verification email (public route)

Method	Endpoint	Description
POST	`/auth/resend-verification-email`	Resend verification email (authenticated route)
POST	`/auth/forgot-password`	Request password reset
POST	`/auth/reset-password`	Reset password

C. Users

Method	Endpoint	Description
GET	`/users`	Get all users
GET	`/users/:id`	Get user by id
PUT	`/users/:id`	Update user
DELETE	`/users/:id`	Delete user

D. Companies

Method	Endpoint	Description
POST	`/companies`	Create company
GET	`/companies`	Get company list
GET	`/companies/:id`	Get company by id
PUT	`/companies/:id`	Update company
DELETE	`/companies/:id`	Delete company
PUT	`/companies/:id/verify`	Verify company (role-restricted)

E. Jobs

Method	Endpoint	Description
POST	`/jobs`	Create job
GET	`/jobs`	Get jobs list (supports filters)
GET	`/jobs/:id`	Get job details
PUT	`/jobs/:id`	Update job
DELETE	`/jobs/:id`	Delete job

F. Applications

Method	Endpoint	Description
POST	`/applications`	Apply to a job
GET	`/applications`	Get applications
GET	`/applications/:id`	Get one application
PUT	`/applications/:id`	Update application status/details
DELETE	`/applications/:id`	Delete application

G. Saved Jobs

Method	Endpoint	Description
POST	`/saved-jobs`	Save a job

Method	Endpoint	Description
GET	`/saved-jobs`	Get saved jobs
DELETE	`/saved-jobs/:id`	Remove saved job

H. Admin

Method	Endpoint	Description
GET	`/admin/users`	Get all users (admin scope)
GET	`/admin/companies`	Get all companies (admin scope)
PUT	`/admin/companies/:id/verify`	Verify company (admin)
PUT	`/admin/users/:id/role`	Change user role

---

6) Screenshots (Placeholders)

> Replace each placeholder with actual screenshots from your running project. Tip: Keep each screenshot with a short caption (what the page shows) for non-technical evaluators.

Screenshot 1: Home Page

[Insert Screenshot Here: Home Page]

Screenshot 2: Jobs Listing Page (with Filters)

[Insert Screenshot Here: Jobs Page]

Screenshot 3: Job Details + Apply Section

[Insert Screenshot Here: Job Detail Page]

Screenshot 4: Companies Listing Page

[Insert Screenshot Here: Companies Page]

Screenshot 5: Company Details Page

[Insert Screenshot Here: Company Detail Page]

Screenshot 6: Authentication (Login/Register)

[Insert Screenshot Here: Auth Page]

Screenshot 7: Dashboard (Role-based)

[Insert Screenshot Here: Dashboard Page]

Screenshot 8: Recruiter/Admin Actions (Edit/Delete/Verify)

[Insert Screenshot Here: Management Actions]

Screenshot 9: Architecture / Flowchart Page (Optional)

[Insert Screenshot Here: Architecture Diagram Rendered]

---

7) Conclusion

This Job Portal is a full hiring system with role-based access.
Job seeker can search, save, and apply. Recruiter can post and manage jobs. Admin can verify and control platform quality.
Because all modules are connected in one system, work is faster, tracking is better, and user experience is clear and simple.